Code & Branch Coverage Report

L6_L7_41 - Laundry Service
Individual Lab 6 submission | Python testing and coverage

This report documents separate Python test suites for statement coverage and branch coverage of `laundry_service.py`. Both meet the required minimum of 90%. The complete test files appear in the appendices and are embedded as separate Python attachments in this PDF.

100%	100%	91
STATEMENT COVERAGE	BRANCH COVERAGE	TEST CASES PASSED

Verified results

Suite / file	Passed	Statements	Branch outcomes
test_code_coverage.py	20	37 / 37 (100%)	Not measured
test_branch_coverage.py	71	37 / 37 (100%)	26 / 26 (100%)

Statement coverage: 37 executed statements / 37 executable statements = 100%.
Branch coverage: 26 executed branch outcomes / 26 measured outcomes = 100%.
Coverage is scoped to the supplied module. The test files and coverage helper are not included in the application coverage denominator.

Submission files and environment

`test_code_coverage.py` - independent code coverage tests.
`test_branch_coverage.py` - independent branch coverage tests.
`screenshots/` and `reports/` - terminal evidence, JSON results and transcripts.
Python 3.12.10 | pytest 9.2.0 | Coverage.py 7.17.0

Repository and separate commits

github.com/ayaan193/SE-labs/tree/main/LAB6
fce88bb - code coverage tests and evidence.
18b08e3 - branch coverage tests and evidence.
Later commits replace the initial captures with the student-provided Windows terminal screenshots and update matching reports.

Test design & execution

The module calculates extra-weight charges, determines whether an order can be accepted, and classifies a drop-off as accepted, late or rejected. The supplied module is kept unchanged; tests assert concrete outputs and expected exceptions.

Area	Cases exercised
Input validation	Unknown/empty garment types and negative weights; expected ValueError messages.
Membership allowance	Regular and delicate membership bonuses, plus the bedding exception.
Charges and surcharges	Included-weight boundary, fractional weights, standard/express rates, exactly ten extra kg and greater than ten extra kg.
Order acceptance	Delicate item limit of five; regular 25 kg and 40-item thresholds, including values on both sides.
Drop-off cutoffs	All four bedding/express combinations, with accepted, late and rejected boundaries at cutoff, cutoff-20 and cutoff-21.

Reproduce the results

Run from `SE-labs/LAB6` after creating the virtual environment and installing `requirements.txt`:

```
.\.venv\Scripts\python.exe run_coverage.py code
.\.venv\Scripts\python.exe run_coverage.py branch
```

Each command runs only its own test file and writes a separate coverage database, JSON report and terminal transcript. The branch run enables `--branch` and independently checks the branch-only percentage from the JSON counts; the ordinary coverage table combines statement and branch opportunities.

Evidence and interpretation

Figures 1-4 reproduce the supplied Windows terminal screenshots without removing their warnings. The screenshots show 20 code tests and 71 branch tests passing, and final coverage of 100%. The visible pytest warning concerns permission to write its optional cache; it does not indicate a failed test.

Result: both suites exceed the required 90% coverage. These results establish execution coverage for this module and these test inputs; they do not imply that every possible input or specification issue has been tested.

Code coverage - terminal run

Figure 1. Separate code coverage run: 20 tests passed, 37 of 37 statements covered, final statement coverage 100%.

```
C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6>.\.venv\Scripts\python.exe run_coverage.py code
LAB 6 - CODE (STATEMENT) COVERAGE
Python 3.12.10

LAB6> python -m coverage run --data-file=.coverage.code --source=laundry_service -m pytest test_code_coverage.py -q
....................                                                                                [100%]
============================================= warnings summary =============================================
.venv\Lib\site-packages\_pytest\cacheprovider.py:487
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWarning: ca
che could not write path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.pytest_cache\v\cache\nodeids: [Errno 13] Permission
denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB6\\.pytest_cache\\v\\cache\\nodeids'
    config.cache.set("cache/nodeids", sorted(str(n) for n in self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
20 passed, 1 warning in 0.06s
LAB6> python -m coverage report --data-file=.coverage.code --show-missing --fail-under=90
Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
laundry_service.py      37      0   100%
-------------------------------------------------
TOTAL                   37      0   100%
LAB6> python -m coverage json --data-file=.coverage.code --pretty-print -o reports/code_coverage.json
Wrote JSON report to reports/code_coverage.json

Statement coverage: 100.00% (37/37 statements)
PASS - minimum required: 90%
```

Source: screenshots/code_coverage.png

Branch coverage - terminal run

Figure 2. Separate branch coverage run: 71 tests passed, 26 of 26 branch outcomes covered, final branch coverage 100%.

```
C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6>.\.venv\Scripts\python.exe run_coverage.py branch
LAB 6 - BRANCH COVERAGE
Python 3.12.10

LAB6> python -m coverage run --data-file=.coverage.branch --branch --source=laundry_service -m pytest test_branch_coverage.py -q
.....................................................................                    [100%]
=============================================== warnings summary ===============================================
.venv\Lib\site-packages\_pytest\cacheprovider.py:487
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWarning: ca
che could not write path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.pytest_cache\v\cache\nodeids: [Errno 13] Permission
denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB6\\.pytest_cache\\v\\cache\\nodeids'
    config.cache.set("cache/nodeids", sorted(str(n) for n in self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
71 passed, 1 warning in 0.18s
LAB6> python -m coverage report --data-file=.coverage.branch --show-missing --fail-under=90
Name                Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------
laundry_service.py     37      0     26      0   100%
---------------------------------------------------------------
TOTAL                  37      0     26      0   100%
LAB6> python -m coverage json --data-file=.coverage.branch --pretty-print -o reports/branch_coverage.json
Wrote JSON report to reports/branch_coverage.json

Statement coverage: 100.00% (37/37 statements)
Branch coverage: 100.00% (26/26 branch outcomes)
PASS - minimum required: 90%
```

Source: screenshots/branch_coverage.png

Code coverage - saved report

Figure 3. Latest code coverage report: 100% statement coverage and confirmation that the JSON report was saved. The next branch command is visible below the code results.

```
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWarning: ca
che could not write path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.pytest_cache\v\cache\nodeids: [Errno 13] Permission
denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB6\\.pytest_cache\\v\\cache\\nodeids'
    config.cache.set("cache/nodeids", sorted(str(n) for n in self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
20 passed, 1 warning in 0.07s
LAB6> python -m coverage report --data-file=.coverage.code --show-missing --fail-under=90
Name                 Stmts   Miss  Cover   Missing
--------------------------------------------------
laundry_service.py      37      0   100%
--------------------------------------------------
TOTAL                   37      0   100%
LAB6> python -m coverage json --data-file=.coverage.code --pretty-print -o reports/code_coverage.json
Wrote JSON report to reports/code_coverage.json

Statement coverage: 100.00% (37/37 statements)
PASS - minimum required: 90%

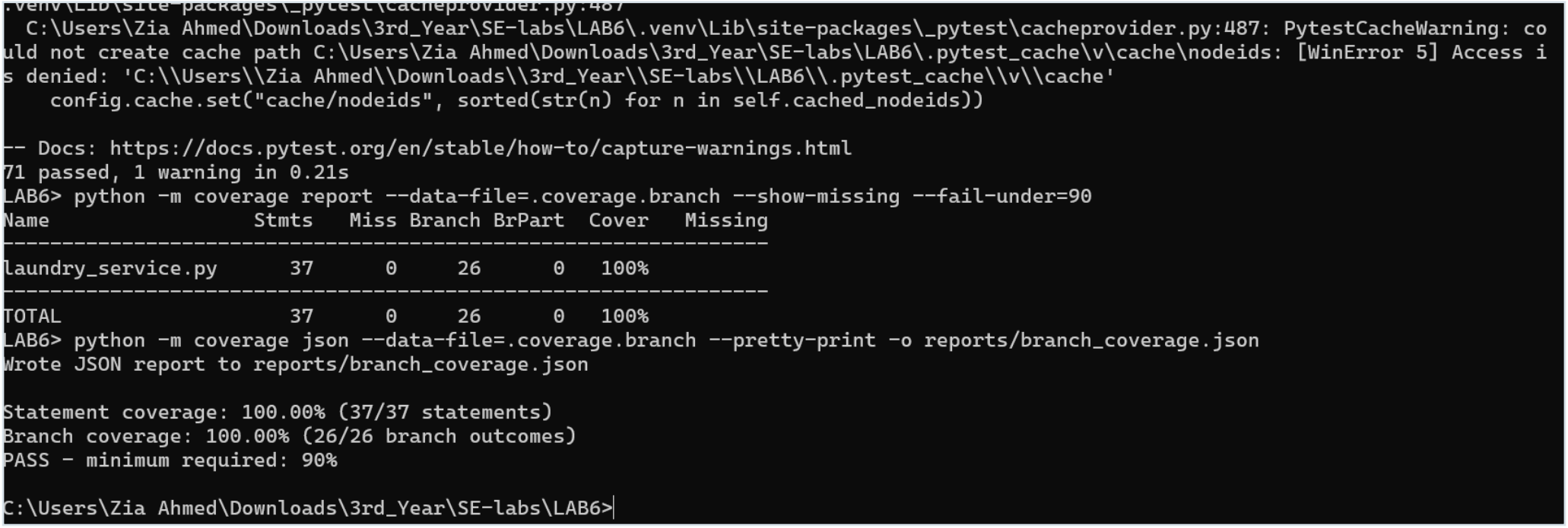
C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6>.\.venv\Scripts\python.exe run_coverage.py branch
LAB 6 - BRANCH COVERAGE
Python 3.12.10

LAB6> python -m coverage run --data-file=.coverage.branch --branch --source=laundry_service -m pytest test_branch_coverage.py -q
.....................................................................                                    [100%]
============================================= warnings summary =============================================
.venv\Lib\site-packages\_pytest\cacheprovider.py:487
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWarning: co
```

Source: reports/code_coverage_screenshot.png

Branch coverage - saved report

Figure 4. Latest branch report: 100% statement coverage, 100% branch coverage and confirmation that the JSON report was saved.



Source: reports/branch_coverage_screenshot.png

Appendix A - Code coverage tests

test_code_coverage.py | full source listing | lines 1-55 of 81

```
"""Statement coverage: exercise every executable line of laundry_service."""

import pytest

from laundry_service import (
    dropoff_status,
    extra_weight_charge,
    order_acceptable,
)


def test_unknown_garment_is_rejected():
    with pytest.raises(ValueError, match="^unknown garment type$"):
        extra_weight_charge("leather", 2, "standard", False)


def test_negative_weight_is_rejected():
    with pytest.raises(ValueError, match="^invalid weight$"):
        extra_weight_charge("regular", -1, "standard", False)


def test_weight_inside_allowance_has_no_charge():
    assert extra_weight_charge("regular", 4, "standard", False) == 0


def test_membership_adds_two_kilos_for_delicates():
    assert extra_weight_charge("delicate", 5, "express", True) == 0


def test_standard_extra_weight_charge():
    assert extra_weight_charge("regular", 7, "standard", False) == 60


def test_express_extra_weight_charge():
    assert extra_weight_charge("regular", 7, "express", False) == 100


def test_fractional_extra_weight_charge():
    assert extra_weight_charge("regular", 5.5, "standard", False) == 15


def test_large_standard_order_uses_surcharge():
    # 16 - 5 = 11 extra kg; standard rate becomes 30 + 15.
    assert extra_weight_charge("regular", 16, "standard", False) == 495


def test_large_express_order_uses_both_surcharges():
    # 16 - 5 = 11 extra kg; express rate becomes 50 + 15 + 10.
    assert extra_weight_charge("regular", 16, "express", False) == 825


def test_bedding_does_not_receive_membership_allowance():
    assert extra_weight_charge("bedding", 9, "standard", True) == 30


```

Appendix A - Code coverage tests

test_code_coverage.py | full source listing | lines 56-81 of 81

```
56  @pytest.mark.parametrize(
57      "weight, items, delicate, expected",
58      [
59          (3, 5, True, True),
60          (3, 6, True, False),
61          (26, 1, False, False),
62          (25, 40, False, True),
63          (10, 41, False, False),
64      ],
65  )
66  def test_order_acceptance(weight, items, delicate, expected):
67      assert order_acceptable(weight, items, delicate) is expected
68
69
70  @pytest.mark.parametrize(
71      "minutes, bedding, express, expected",
72      [
73          (90, True, False, "accepted"),
74          (90, False, True, "accepted"),
75          (60, False, False, "accepted"),
76          (45, False, False, "late"),
77          (39, False, False, "rejected"),
78      ],
79  )
80  def test_dropoff_outcomes(minutes, bedding, express, expected):
81      assert dropoff_status(minutes, bedding, express) == expected
```

Appendix B - Branch coverage tests

test_branch_coverage.py | full source listing | lines 1-44 of 120

```
"""Branch coverage: independently exercise decisions and their boundaries."""

import pytest

from laundry_service import (
    dropoff_status,
    extra_weight_charge,
    order_acceptable,
)


@pytest.mark.parametrize("garment", ["silk", "", None])
def test_unknown_garment_branch(garment):
    with pytest.raises(ValueError, match="^unknown garment type$"):
        extra_weight_charge(garment, 0, "standard", False)


@pytest.mark.parametrize("weight", [-0.01, -1, -100])
def test_negative_weight_branch(weight):
    with pytest.raises(ValueError, match="^invalid weight$"):
        extra_weight_charge("regular", weight, "standard", False)


def test_garment_validation_precedes_weight_validation():
    with pytest.raises(ValueError, match="^unknown garment type$"):
        extra_weight_charge("silk", -1, "standard", False)


@pytest.mark.parametrize(
    "garment, member, expected",
    [
        ("regular", False, 90),
        ("regular", True, 30),
        ("delicate", False, 150),
        ("delicate", True, 90),
        ("bedding", False, 0),
        ("bedding", True, 0),
    ],
)
def test_membership_condition_combinations(garment, member, expected):
    # Covers both short-circuit outcomes and the bedding membership exception.
    assert extra_weight_charge(garment, 8, "standard", member) == expected


```

Appendix B - Branch coverage tests

test_branch_coverage.py | full source listing | lines 45-90 of 120

```
45  @pytest.mark.parametrize(
46      "garment, member, allowance",
47      [
48          ("regular", False, 5),
49          ("regular", True, 7),
50          ("delicate", False, 3),
51          ("delicate", True, 5),
52          ("bedding", False, 8),
53          ("bedding", True, 8),
54      ],
55  )
56  @pytest.mark.parametrize("extra, expected", [(-0.5, 0), (0, 0), (0.5, 15)])
57  def test_included_weight_boundary(garment, member, allowance, extra, expected):
58      assert extra_weight_charge(
59          garment, allowance + extra, "standard", member
60      ) == expected
61
62
63  @pytest.mark.parametrize(
64      "weight, speed, expected",
65      [
66          (0, "standard", 0),
67          (5.5, "standard", 15),
68          (5.5, "express", 25),
69          (15, "standard", 300),
70          (15, "express", 500),
71          (15.5, "standard", 472.5),
72          (15.5, "express", 787.5),
73      ],
74  )
75  def test_rate_and_ten_extra_kilo_boundary(weight, speed, expected):
76      # Exactly ten extra kg has no surcharge; above ten applies it to all extra kg.
77      assert extra_weight_charge("regular", weight, speed, False) == pytest.approx(
78          expected
79      )
80
81
82  @pytest.mark.parametrize(
83      "weight, items, expected",
84      [(1, 4, True), (1, 5, True), (1, 6, False), (100, 5, True)],
85  )
86  def test_delicate_item_limit_and_weight_bypass(weight, items, expected):
87      # Current specification checks only item count for delicate orders.
88      assert order_acceptable(weight, items, True) is expected
89
90
```

Appendix B - Branch coverage tests

test_branch_coverage.py | full source listing | lines 91-120 of 120

```
91  @pytest.mark.parametrize(
92      "weight, items, expected",
93      [
94          (24.99, 39, True),
95          (24.99, 40, True),
96          (24.99, 41, False),
97          (25, 39, True),
98          (25, 40, True),
99          (25, 41, False),
100         (25.01, 39, False),
101         (25.01, 40, False),
102         (25.01, 41, False),
103     ],
104 )
105 def test_regular_weight_and_item_boundaries(weight, items, expected):
106     assert order_acceptable(weight, items, False) is expected
107
108
109 @pytest.mark.parametrize(
110     "bedding, express, cutoff",
111     [(False, False, 60), (False, True, 90), (True, False, 90), (True, True, 120)],
112 )
113 @pytest.mark.parametrize(
114     "offset, expected",
115     [(1, "accepted"), (0, "accepted"), (-1, "late"), (-20, "late"), (-21, "rejected")],
116 )
117 def test_all_dropoff_decisions_and_cutoff_boundaries(
118     bedding, express, cutoff, offset, expected
119 ):
120     assert dropoff_status(cutoff + offset, bedding, express) == expected
```